

Truly Functional Solutions to the Longest Uptrend Problem (Functional Pearl)

ALEXANDER DINGES, RPTU Kaiserslautern-Landau, Germany

RALF HINZE, RPTU Kaiserslautern-Landau, Germany

Solutions to the longest increasing subsequence problem are typically implemented imperatively, relying on arrays for constant-time lookups and updates. Replacing these arrays with functional sequences allows a purely functional solution with the same asymptotic running time, but with significantly worse practical performance. In this pearl, we present a purely functional approach that is not only asymptotically optimal, but also efficient in practice. The core idea is to exploit the interplay between search, lookup, and update operations through Huet’s zipper. In addition, we improve the adaptive behaviour of imperative solutions commonly found in the literature.

CCS Concepts: • **Theory of computation** → *Data structures design and analysis*; • **Software and its engineering** → **Functional languages**.

Additional Key Words and Phrases: Longest Increasing Subsequence, Zipper, Haskell

ACM Reference Format:

Alexander Dinges and Ralf Hinze. 2025. Truly Functional Solutions to the Longest Uptrend Problem (Functional Pearl). *Proc. ACM Program. Lang.* 9, ICFP, Article 251 (August 2025), 16 pages. <https://doi.org/10.1145/3747520>

1 Introduction

I can resist anything except temptation.

Oscar Wilde, Lady Windermere’s Fan

“Shall I be pure or impure?” Many temptations lure the functional programmer: input/output, exceptions, references, and, in particular, arrays with constant-time update. And the sirens whisper: “Haskell is the world’s finest imperative programming language.” We openly admit that we too have succumbed to temptation: on more than one occasion, the efficiency promises of updatable arrays have been too compelling. Here is a recent example.

We derived an algorithm for solving the “longest increasing subsequence” problem. Briefly, given a sequence of elements, the problem is to find a longest subsequence that is strictly increasing. A subsequence is given by a selection of the original data, such that the relative order of the data items is preserved. The derived algorithm can justifiably be called imperative, and indeed we used the *ST*-monad for the Haskell implementation. For reference, Figure 1 summarizes the most important facts, using pseudo-code rather than monadic code to emphasize its imperative nature. Indeed, open a classic textbook on program derivation [2, 5] and you will find similar solutions. The central operation is an in-place array update:

$$\begin{aligned} i &\leftarrow \text{BINARY-SEARCH} \dots \\ s[i - 1] &\leftarrow a : s[i] \end{aligned}$$

Authors’ Contact Information: Alexander Dinges, RPTU Kaiserslautern-Landau, Kaiserslautern, Germany, alexander.dinges@cs.rptu.de; Ralf Hinze, RPTU Kaiserslautern-Landau, Kaiserslautern, Germany, ralf.hinze@cs.rptu.de.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 2475-1421/2025/8-ART251

<https://doi.org/10.1145/3747520>

The pseudocode that follows takes as input a list xs and it returns a longest increasing subsequence of xs (longest uptrend for short) together with its length. The program maintains a table of candidate solutions: $s[i]$ is an uptrend of length $n - i$. The *extensibility* relation “ $\prec$ ” specifies when a candidate can be extended at the front: $a \prec w = (\text{null } w) \vee (a < \text{head } w)$.

LONGEST-INCREASING-SUBSEQUENCE (xs)

```

1   $n \leftarrow \text{LENGTH}(xs)$ 
2   $m \leftarrow n$ 
3   $s[n] \leftarrow []$ 
4  for  $a$  in REVERSE ( $xs$ )
5      do if  $a \prec s[m]$ 
6          then  $s[m - 1] \leftarrow a : s[m]$            ▷ grow table by one entry
7               $m \leftarrow m - 1$ 
8          else  $i \leftarrow \text{BINARY-SEARCH}(\lambda j. a \prec s[j]) (m + 1, n)$ 
9               $s[i - 1] \leftarrow a : s[i]$            ▷ update an existing entry
10 return  $s[m]$  and  $n - m$ 
```

For each element of the input, the table is either grown or an existing entry is updated. If the current longest uptrend at position m can be extended, a longer uptrend has been found. Otherwise, a binary search determines the least i such that $s[i]$ is extensible. The uptrend $a : s[i]$ improves $s[i - 1]$, which is accordingly updated. We say that w improves v if every extension of v also extends w , formally: $v \preccurlyeq w \iff \forall a. (a \prec v) \Rightarrow (a \prec w)$. The candidates are not only arranged in *decreasing* order of length, but also in *increasing* order of improvement — the shorter, the better: $i \leq j \implies s[i] \preccurlyeq s[j]$. The program maintains this invariant.

An example run of the algorithm is shown in Figure 2.

Fig. 1. An $n \cdot \log m$ algorithm for solving the “longest increasing subsequence” problem.

The details are not important. Suffice it to say that a binary search determines an index, which is then used to read one array element and to update a second. To functionalize the code, we can replace the ephemeral, imperative array with a persistent, functional array, but you might argue that this representation change does not remove the imperative nature.

```

let  $i = \text{binary-search} \dots$ 
in  $\text{update } s (i - 1) (a : s ! i)$ 
```

If each operation — search, look-up, and update — runs in logarithmic time, then the asymptotic running time is no worse than that of the imperative fragment. However, the constant factors are worse, as look-up and update do not run in constant-time; and, indeed, benchmarks demonstrate the superiority of the imperative program.

We gave up ... but the problem gnawed at us. After a while, we realized that we had been guilty of imperative thinking. The three operations — search, look-up, and update — are closely intertwined, but they communicate rather indirectly via numerical positions, via indices. Surely there is a better way of arranging their interplay. This pearl tells the story of a truly functional solution to the “longest increasing subsequence” problem (“longest uptrend” for short). The motto is “*Down with indices!*”

Section 2 introduces the basic idea using the data-structural manifestation of binary search: binary trees. Section 3 addresses the inefficiency of binary trees by introducing a balancing scheme. Finally, Section 4 improves the interface and the implementation.

This paper is aimed at readers with a basic understanding of functional programming; no specialized knowledge of algorithm design is presumed.

Given the input sequence $[0, 7, 4, 6, 1, 5, 3, 4, 1]$ of length 9, the algorithm returns $[0, 1, 3, 4]$ as its longest increasing subsequence. It starts with an array s of length 10, setting $s[9] = []$. The input sequence is processed in reverse order, updating the array s as follows:

									$[]$	1
								$[1]$	$[]$	4
								$[4]$	$[]$	3
							$[3, 4]$	$[4]$	$[]$	5
							$[3, 4]$	$[5]$	$[]$	1
						$[1, 3, 4]$	$[3, 4]$	$[5]$	$[]$	6
						$[1, 3, 4]$	$[3, 4]$	$[6]$	$[]$	4
						$[1, 3, 4]$	$[4, 6]$	$[6]$	$[]$	7
						$[1, 3, 4]$	$[4, 6]$	$[7]$	$[]$	0
					$[0, 1, 3, 4]$	$[1, 3, 4]$	$[4, 6]$	$[7]$	$[]$	

The active part of the array is highlighted in orange. In each step, either a longer uptrend is added at the front or an existing candidate is replaced by an improved one. Only in the latter case, a binary search is performed.

Observe that the candidates in each row (and each column) are arranged in increasing order of improvement.

Fig. 2. An example run of LONGEST-INCREASING-SUBSEQUENCE.

2 Warm Up: Unbalanced, Binary Trees

2.1 Binary Trees

We represent sequences by binary, node-labelled trees.

```
data Tree a = Leaf
            | Node (Tree a) a (Tree a)
```

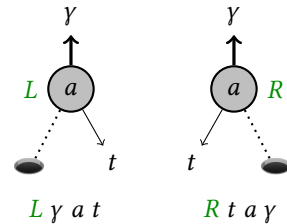
```
single :: a → Tree a
```

```
single a = Node Leaf a Leaf
```

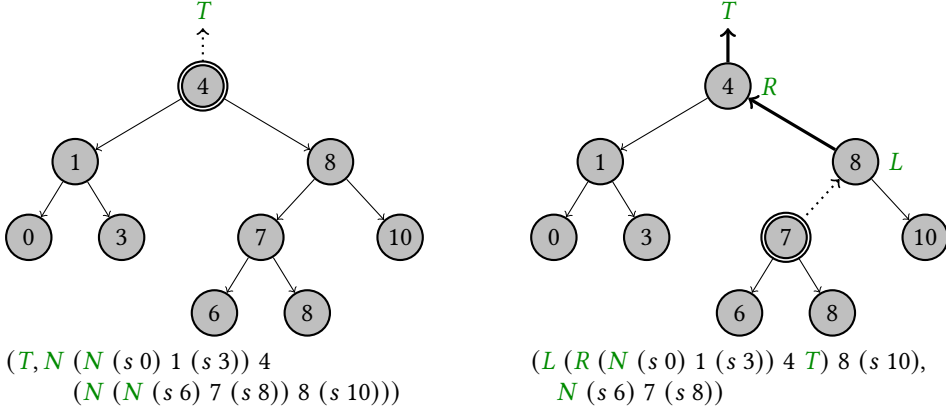
2.2 Zipper

Since we frown upon numerical positions, we must first find a suitable replacement. Fortunately, we do not have to look far: locations within a tree can be *functionally* represented using Huet's zipper (1997).

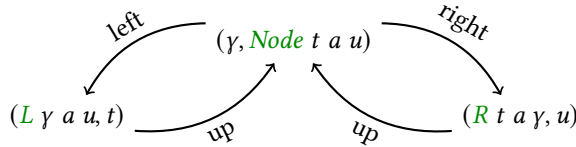
```
data Context a
  = T
  | L (Context a) a (Tree a)
  | R (Tree a) a (Context a)
type Loc a = (Context a, Tree a)
```



Roughly speaking, the zipper reverses the “pointers” along a path, replacing downward pointers (marked by a hole $\bullet$) with upward pointers (drawn as bold edges). The constructors L and R signal that the hole is located on the left or on the right. We use dashed edges to depict the position of the hole. To illustrate, consider the two locations below (N is short for *Node* and s for *single*).



A location consists of a context and a tree. The context is the path leading to the top; the tree is the current focus of attention (its root is marked by a double circle). Starting from the root, we make two moves: first to the right and then to the left. A move changes the location in the following way.



Since we never lose any information, each move can be undone. If we always move up, zipping the context with the tree, we re-create the original tree (do not confuse *zip* with its namesake on lists).

$$\begin{aligned}
 \text{zip} &:: \text{Loc } a \rightarrow \text{Tree } a \\
 \text{zip } (T, \quad r) &= r \\
 \text{zip } (L \gamma a u, t) &= \text{zip } (\gamma, \text{Node } t a u) \\
 \text{zip } (R t a \gamma, u) &= \text{zip } (\gamma, \text{Node } t a u)
 \end{aligned}$$

2.3 Binary Search

Given a predicate p and a sequence s represented by a tree, binary search returns **some** tipping point, a pair of adjacent elements in s , say a and b , such that $\neg p a$ and $p b$. (In Figure 1, binary search returns the index of b .) Of course, we do not actually return the elements, but their locations, for example, $(\gamma, \text{Node } t a u)$ and $(\delta, \text{Node } v b w)$; the element is the root label of the focus. We must also consider the case that the sequence s is empty, and that the tipping point is on the far left or on the far right — a case for *Maybe*.

```

binary-search :: (a → Bool) → Tree a → (Maybe (Loc a), Maybe (Loc a))
binary-search p s = find Nothing (T, s) Nothing
  where find :: Maybe (Loc a) → Loc a → Maybe (Loc a) → (Maybe (Loc a), Maybe (Loc a))
        find neg cur@(γ, Leaf) pos = (neg, pos)
        find neg cur@(γ, Node l x r) pos
          | p x      = find neg      (L γ x r, l) (Just cur)
          | otherwise = find (Just cur) (R l x γ, r) pos

```

The worker function *find* maintains pointers to a candidate for *a* (*neg*), to the current location (*cur*), and to a candidate for *b* (*pos*). The wrapper function initializes the two pointers to *Nothing*, which can be seen as representing the boundaries of the sequence *s*. At any point during the recursion, the current focus contains exactly the elements between *a* and *b*. Consequently, if the focus is an empty leaf, two adjacent elements are found. Moreover, *find* maintains the invariant that $\neg p \text{ a and } p \text{ b}$ (if applicable). Note that we do not assume that the sequence *s* is ordered, nor do we assume any properties of the predicate *p*. (A common view is that binary search is about searching an ordered table — the recent pearl by Dinges and Hinze debunks this myth.)

2.4 Longest Uptrend

As explained in Figure 1, the algorithm for solving the “longest uptrend” problem maintains a sequence of candidate solutions arranged in *decreasing* order of length. Each candidate is the best uptrend (with respect to the improvement relation $\preccurlyeq$) among all uptrends of the same length. When every input element has been processed, the leftmost candidate is therefore the longest uptrend (the definition of *leftmost* is standard and omitted).

```

best-longest-uptrend :: (Ord a) ⇒ [a] → [a]
best-longest-uptrend = leftmost · foldr insert (single [])

```

The best and only uptrend of length 0 is the empty list $[]$. The step function *insert* first determines a tipping point with respect to the extensibility relation: two adjacent uptrends, say *w* and *v*, such that $\neg a \prec w$ and $a \prec v$. Depending on the location of the tipping point, *insert* then takes one of two actions.

```

insert :: (Ord a) ⇒ a → Tree [a] → Tree [a]
insert a t = case binary-search (a <=) t of
  (Nothing, Just (γ, Node Leaf v r)) → zip (γ, Node (single (a : v)) v r)
  (Just (γ, Node l w r), Just (⊥, Node ⊥ v ⊥)) → zip (γ, Node l (a : v) r)

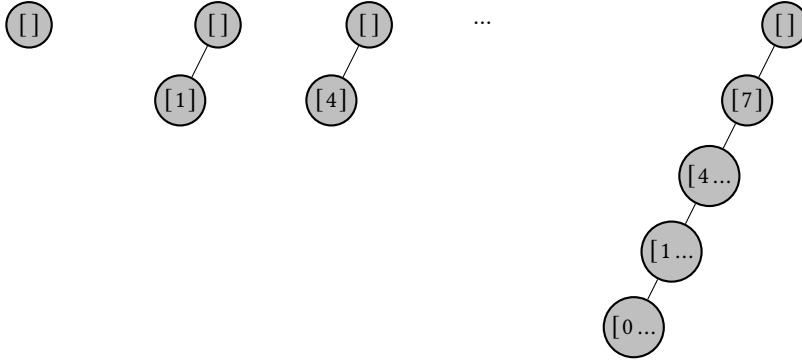
```

If the tipping point is on the far left, then the very first uptrend *v* is extensible. The new uptrend $a : v$ is added to the sequence by replacing the empty left subtree with a singleton tree. Otherwise, the uptrend *w* can be improved and is replaced by $a : v$. In each case, the updated location is zipped to a tree. The patterns are not exhaustive, but this is no cause for alarm: the second location is necessarily proper, as the empty uptrend is trivially extensible.

3 Balanced Binary Tree

The implementation in Section 2.4 serves mainly as a proof of concept. It has a quadratic worst-case running time $\Theta(n^2)$, because the binary tree always degenerates to a list-like structure, see Figure 3. However, a fix is easy to find: we impose a balancing scheme. There are a few dozen options to choose from. Since our trees only grow, never shrink, we pick the simplest, and therefore most attractive scheme: Okasaki’s red-black trees (1999). If you need a refresher, Figure 4 summarizes the key facts about red-black trees.

Continuing the example from Figure 2, *best-longest-uptrend* applied to the input $[0, 7, 4, 6, 1, 5, 3, 4, 1]$ produces the following sequence of trees (from left to right):



Each tree corresponds to a row in Figure 2. Sadly, since the trees are always extended on the left, they degenerate to a list-like structure. Thus, *binary-search* is, in fact, a linear search.

Fig. 3. An example run for *best-longest-uptrend*.

3.1 Red-Black Trees

The changes are minor. The nodes of binary trees are augmented with a colour field.

```
data Colour = Red | Black
data Red-Black-Tree a = Leaf
    | Node Colour (Red-Black-Tree a) a (Red-Black-Tree a)
```

We take the opportunity to introduce a few helper functions.

```
new :: a → Red-Black-Tree a
new a = Node Red Leaf a Leaf
red4 :: Red-Black-Tree a → a → Red-Black-Tree a → a
      → Red-Black-Tree a → a → Red-Black-Tree a → Red-Black-Tree a
red4 t a u b v c w = Node Red (Node Black t a u) b (Node Black v c w)
```

The code of Sections 2.2 and 2.3 is easily adapted and hence omitted: we only need to add colour fields to the constructors *Node*, *L*, and *R*.

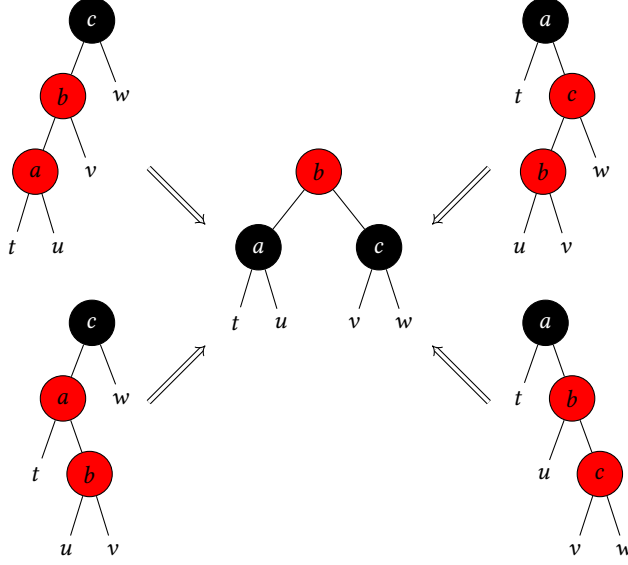
3.2 Longest Uptrend – encore une fois

The implementation of *best-longest-uptrend* is also unchanged, with one notable exception: if a new node is added, we possibly need to re-balance the tree, see the first case below.

```
insert :: (Ord a) ⇒ a → Red-Black-Tree [a] → Red-Black-Tree [a]
insert a t = case binary-search (a <) t of
    (Nothing, Just (γ, Node c Leaf v r)) → balance (L c γ v r, new (a : v))
    (Just (γ, Node c l w r), Just (–, Node – v –)) → zip (γ, Node c l (a : v) r)
```

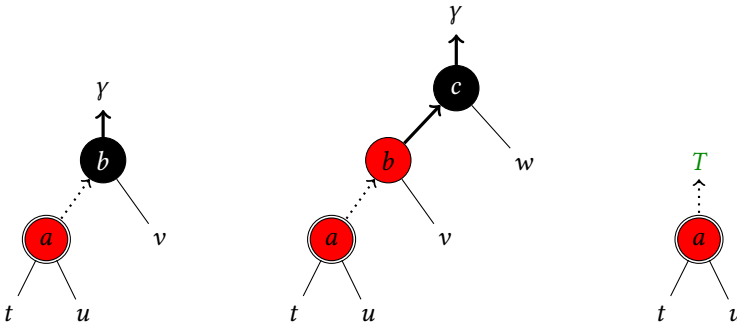
The helper function *balance* takes a location and maintains the invariant that the focus is coloured red. It walks *up* the left spine, distinguishing three cases:

Red-black trees are a cunning representation of 2-3-4 trees, where ternary and quaternary nodes are represented by small binary trees consisting of a black root and one or two red children. The relation to 2-3-4 trees motivates the *red-condition*: a red node can only appear as a child of a black node. 2-3-4 trees are perfectly balanced, motivating the *black-condition*: on any path from the root to a leaf, the number of black nodes is the same — the number is called the *black-height*. The two conditions ensure that each path from the root to a leaf has logarithmic length in the total number of nodes.



Insertion works in the same way as with binary trees, except that a leaf is replaced by a *red* node, possibly violating the red-condition, but not the black-condition. The violation is fixed by local transformations along the search path, as illustrated above. Each rewrite possibly causes a violation further up the tree. If a red node percolates to the top, it is turned into a black node, increasing the black-height by one.

Fig. 4. Red-black trees: a brief summary.



If the parent is black, no (further) re-balancing is necessary. If the parent is red, the red-condition is violated: the red-red-black path is transformed into a red node with two black children, see Figure 4; and the re-balancing continues. If there is no parent node, the red node has percolated to the top, where it is re-coloured to be black.

```

balance :: Loc a → Red-Black-Tree a
balance (T,
  balance (L Red (L Black γ c w) b v, Node Red t a u) = Node Black t a u
  balance (L Black γ b v, Node Red t a u) = zip (L Black γ b v, Node Red t a u)

```

Again, the patterns are not exhaustive; and again, this is no cause for alarm. Since the path initially points to the new leftmost element, it is necessarily a sequence of *L* constructors ending in a *T*. Huet's zipper has proved its worth. As we are able to walk *up* the tree, violations of the red-condition are straightforward to detect and to repair.

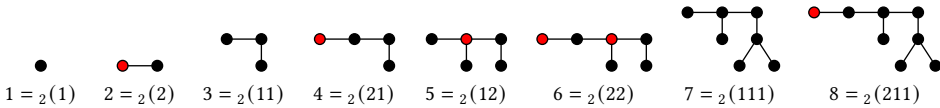
Observe that both *zip* and *balance* are tail-recursive, which is one of the reasons why the implementation of *best-longest-uptrend* performs very well in practice. It is also asymptotically optimal: it runs in $\Theta(n \cdot \log m)$ time, where n is the length of the input and m is the length of a longest uptrend. But there is still room for improvement.

4 Finish: Finger Tree

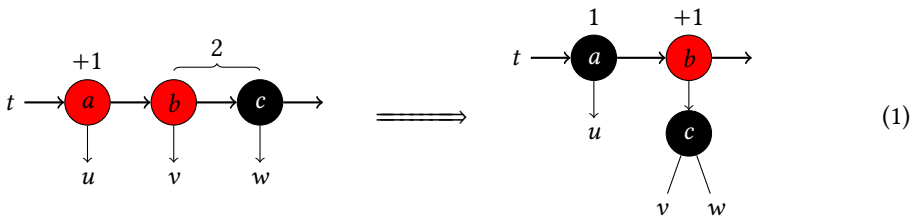
The implementation can be improved in two ways, addressing two orthogonal issues. The first improvement is of algorithmic nature and addresses an efficiency problem. The imperative solution runs in linear time, if the input is sorted or nearly sorted, as the case where the table grows is treated separately (lines 5-7 in Figure 1). To achieve the same *adaptive* behaviour, we need to be able to extend a sequence in amortized constant time and to access the head in constant time. The second improvement concerns a software-engineering issue: the lack of a proper interface. Both the data structures, red-black trees and zippers, and the central algorithm, binary search, are tailored to the application and are of little general use. (The search returns a pointer to a node, revealing implementation details. A malicious adversary could tinker with the internals. Ouch.) We address both problems in turn.

4.1 Left-Spine View

Let us start with a crucial observation. Our application is rather special in that new elements are only added at the front. This usage pattern has an interesting consequence: *the size of a red-black tree completely determines its shape* [6]. To see why this is the case, let us consider a few small examples. The diagrams below show the shapes for sizes up to 8. For perspicuity, the insertion path, the so-called *left spine*, is drawn horizontally (ignore the numbers for the moment).



We observe that red nodes are rather rare; they only appear *on* the left spine. This is not a coincidence. New nodes, which are coloured red, are placed *on* the left spine. If the red-condition is violated, the first rewrite rule of Figure 4 fires. The rule is repeated below, but with a twist: in accordance with the examples, the left spine is drawn horizontally.



The rule maintains the invariant that red nodes only appear *on* the spine. Consequently, the trees hanging from the spine are all black. This has two immediate consequences. First, only the first rule ever fires, see [Figure 4](#). Second, the black-condition entails that the trees below the spine are perfectly balanced. Thus, the left spine is a sequence of topped perfect trees, so-called *pennants*. The black-condition also implies that a pennant of height h appears either once or twice. Since the size of a pennant of height h is 2^h , the shape of a tree of size n is uniquely determined by the binary representation of n :

$$_2(d_0 \dots d_{k-1}) := \sum_{i=0}^{k-1} d_i \cdot 2^i \quad \text{with } d_i \in \{1, 2\}.$$

The digit 1 corresponds to a single black pennant, the digit 2 to a red pennant followed by a black pennant, see the examples above. (Note that the digit 0 is not needed in this context.) The weight of the digit equals the size of the pennant(s). Taking the analogy further, the rewrite rule (1) can be seen as implementing a carry, mapping $1 \cdot 2^i + 2 \cdot 2^i$ to $3 \cdot 2^i = 1 \cdot 2^i + 1 \cdot 2^{1+i}$.

The analysis suggests a representation change, switching from the standard representation of red-black trees to the *left-spine view*.

```
data Digit = One | Two
data Spine a = Nil
             | Cons Digit a (Tree a) (Spine a)
```

Ironically, colours are no longer needed: the digits encode the colour information of the nodes on the spine; the trees hanging from the spine are black anyway. The resulting structure is also known as a (one-sided) *finger tree* — the spine view can be seen as a finger pointing to the leftmost element. Of course, we have come across the *Spine* datatype before. The left-spine view is a special case of a context: if (γ, Leaf) is the location of the leftmost leaf, then γ represents the left spine. In more detail, we have the following correspondences.

T	$\cong$	Nil	
$L \text{ Black } \gamma a t$	$\cong$	$\text{Cons One } a t s$	$\Leftarrow \gamma \cong s$
$L \text{ Red } (L \text{ Black } \gamma b u) a t$	$\cong$	$\text{Cons Two } a (\text{Node } t b u) s$	$\Leftarrow \gamma \cong s$

So, sequences are no longer represented by trees, but morally by zippers!

Finger trees are a so-called *numerical representation*. The container type is modelled after the 1-2 binary number system; operations on finger trees are modelled after arithmetic operations on 1-2 binary numbers.

The empty sequence corresponds to zero, a singleton tree to one,

```
nil :: Spine a
nil = Nil
single :: a → Spine a
single a = Cons One a Leaf Nil
```

and *consing* an element to the binary increment. (The worker function *incr* adapts *balance* to the new representation.)

```
cons :: a → Spine a → Spine a
cons a s = incr a Leaf s

incr :: a → Tree a → Spine a → Spine a
incr a1 t1 (Nil) = Cons One a1 t1 Nil
incr a1 t1 (Cons One a2 t2 s) = Cons Two a1 (Node t1 a2 t2) s
incr a1 t1 (Cons Two a2 t2 s) = Cons One a1 t1 (incr a2 t2 s)
```

Since *cons* is modelled after the binary increment, it runs in amortized constant time.

4.2 Zippifying the Zipper

It remains to adapt the zipper to the new representation. However, there is a small challenge: the new sequence type involves two different types of trees: *Spine a* and *Tree a*. So we need to look at the concept of a context in a more principled way. A context can be seen as a term with a hole; plugging the hole with a term yields another term. In a typed setting, the type of contexts therefore needs to record the type of the hole, say *a*, and the type of the resulting term, say *r*. Contexts are not unlike functions, so we use a similar notation: $a \rightrightarrows r$. The type *Context a* from Section 2.2 corresponds to the more informative type $\text{Tree } a \rightrightarrows \text{Tree } a$.

An *n*-ary constructor gives rise to *n* different contexts, for example,

$$\begin{aligned} K &:: A \rightarrow B \rightarrow C \rightarrow D \\ K_1 &:: \forall r . (D \rightrightarrows r) \rightarrow B \rightarrow C \rightarrow (A \rightrightarrows r) \\ K_2 &:: \forall r . A \rightarrow (D \rightrightarrows r) \rightarrow C \rightarrow (B \rightrightarrows r) \\ K_3 &:: \forall r . A \rightarrow B \rightarrow (D \rightrightarrows r) \rightarrow (C \rightrightarrows r) \end{aligned}$$

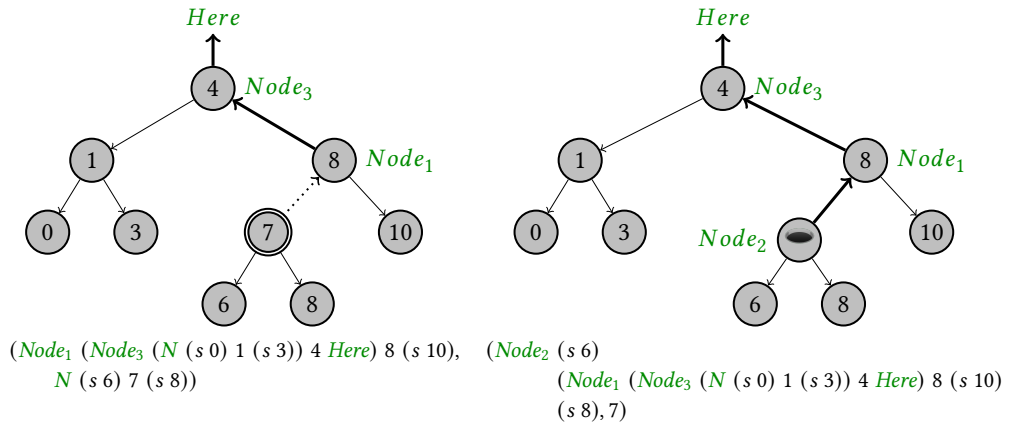
As in Section 2.2, we place the upward pointer of type $D \rightrightarrows r$ in the hole.

For the application at hand, we need contexts for *Nodes* and *Conses*. Since the result types of the constructors differ, they must be introduced using a generalized algebraic datatype.

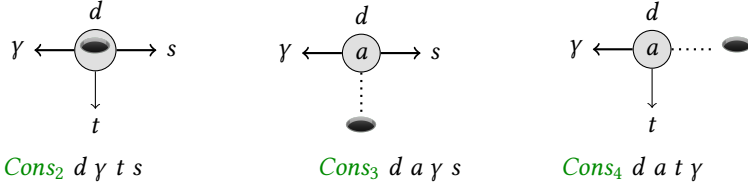
data $a \rightrightarrows b$ where

$$\begin{aligned} \text{Here} &:: \forall r . r \rightrightarrows r \\ \text{Node}_1 &:: \forall r a . (\text{Tree } a \rightrightarrows r) \rightarrow a \rightarrow \text{Tree } a \rightarrow (\text{Tree } a \rightrightarrows r) \\ \text{Node}_2 &:: \forall r a . \text{Tree } a \rightarrow (\text{Tree } a \rightrightarrows r) \rightarrow \text{Tree } a \rightarrow (a \rightrightarrows r) \\ \text{Node}_3 &:: \forall r a . \text{Tree } a \rightarrow a \rightarrow (\text{Tree } a \rightrightarrows r) \rightarrow (\text{Tree } a \rightrightarrows r) \\ \text{Cons}_2 &:: \forall r a . \text{Digit} \rightarrow (\text{Spine } a \rightrightarrows r) \rightarrow \text{Tree } a \rightarrow \text{Spine } a \rightarrow (a \rightrightarrows r) \\ \text{Cons}_3 &:: \forall r a . \text{Digit} \rightarrow a \rightarrow (\text{Spine } a \rightrightarrows r) \rightarrow \text{Spine } a \rightarrow (\text{Tree } a \rightrightarrows r) \\ \text{Cons}_4 &:: \forall r a . \text{Digit} \rightarrow a \rightarrow \text{Tree } a \rightarrow (\text{Spine } a \rightrightarrows r) \rightarrow (\text{Spine } a \rightrightarrows r) \end{aligned}$$

The constructor *Here* corresponds to *T* of the *Context* datatype, *Node*₁ and *Node*₃ to *L* and *R*. The constructor *Node*₂ allows us to put the focus on the content of a node, as illustrated on the right below. For comparison, we have replicated the example of Section 2.2 on the left.



Even though the *Cons* constructor has four arguments, it features only three contexts as we never need to manipulate the first argument, the digit. Using these contexts we can move in both directions on the spine, not unlike a Turing tape. Diagrammatically, if we continue to draw the left spine horizontally, the “upward” pointers now point leftwards.



On an amusing note, recall that the spine is a special case of a zipper, so we are “zippifying” a zipper. One-hole contexts have been characterized as derivatives of datatypes [8]. Hence, our new zipper datatype is a second derivative.

Zipping a typed context with a typed term corresponds closely to function application — hence the name of the function.

$$\begin{aligned}
 \text{apply} &:: (a \Rightarrow b) \rightarrow (a \rightarrow b) \\
 \text{apply } (\text{Here}) & \quad t = t \\
 \text{apply } (\text{Node}_1 \gamma a u) & \quad t = \text{apply } \gamma (\text{Node } t a u) \\
 \text{apply } (\text{Node}_2 t \gamma u) & \quad a = \text{apply } \gamma (\text{Node } t a u) \\
 \text{apply } (\text{Node}_3 t a \gamma) & \quad u = \text{apply } \gamma (\text{Node } t a u) \\
 \text{apply } (\text{Cons}_2 d \gamma t s) & \quad a = \text{apply } \gamma (\text{Cons } d a t s) \\
 \text{apply } (\text{Cons}_3 d a \gamma s) & \quad t = \text{apply } \gamma (\text{Cons } d a t s) \\
 \text{apply } (\text{Cons}_4 d a t \gamma) & \quad s = \text{apply } \gamma (\text{Cons } d a t s)
 \end{aligned}$$

4.3 Exponential Search

Searching a finger tree proceeds in two steps: first, we determine the full binary tree, in which the tipping point is located; then we search this tree. Just as binary trees are the manifestation of *binary search*, (one-sided) finger trees are the incarnation of *exponential search*. (If you are curious, you may want to peek ahead at Figure 5 for an illustration of the correspondence.)

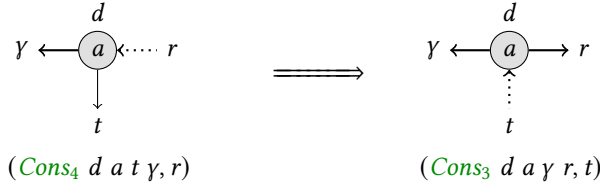
Thanks to the typed zipper, we can have the search return the position of *elements*, rather than nodes. A location of an element within a finger tree has type $(a \Rightarrow \text{Spine } a, a)$. Loosely speaking, the first component allows us to *write* a new value to the location, while the second component allows us to *read* the old value. If we drop one of the components, we obtain *read-only* or *write-only* locations. Tailored to our application, exponential search returns the optional left location write-only and the optional right location read-only.

$$\begin{aligned}
 \text{exponential-search} &:: \forall a. (a \rightarrow \text{Bool}) \rightarrow \text{Spine } a \rightarrow (\text{Maybe } (a \Rightarrow \text{Spine } a), \text{Maybe } a) \\
 \text{exponential-search } p \ (\text{Nil}) &= (\text{Nothing}, \text{Nothing}) \\
 \text{exponential-search } p \ (\text{Cons } d a t r) & \\
 \quad | p a &= (\text{Nothing}, \text{Just } a) \\
 \quad | \text{otherwise} &= \text{explore } (\text{Cons}_4 d a t \text{Here}, r)
 \end{aligned}$$

The wrapper function deals with two simple cases: the finger tree is empty; or the leftmost element already satisfies the predicate. If neither case applies, we start exploring.

where $explore :: (Spine\ a \Rightarrow r, Spine\ a) \rightarrow (Maybe\ (a \Rightarrow r), Maybe\ a)$
 $explore\ cur@(y, Nil) = rewind\ cur\ Nothing$
 $explore\ cur@(y, Cons\ e\ b\ u\ s)$
 $\quad | p\ b = rewind\ cur\ (Just\ b)$
 $\quad | otherwise = explore\ (Cons_4\ e\ b\ u\ y, s)$
 $rewind :: (Spine\ a \Rightarrow r, Spine\ a) \rightarrow Maybe\ a \rightarrow (Maybe\ (a \Rightarrow r), Maybe\ a)$
 $rewind\ (Cons_4\ d\ a\ t\ y, r)\ pos = find\ (Cons_2\ d\ y\ t\ r)\ (Cons_3\ d\ a\ y\ r, t)\ pos$

The worker function assumes that the previous element on the spine violates the predicate. It walks along the spine until it finds an element that satisfies it, or until it reaches the end. In both cases, a binary search is subsequently performed using *find*, introduced further below. Note that the argument of *explore* is a location on the spine, which enables us to move in both directions. The function *rewind* makes good use of this feature.



It moves one position to the left, changing the focus from the spine *r* to the tree *t* that is hanging from the spine, and then passes control to *find*.

Apart from minor adjustments, the implementation of binary search is identical to the code from Section 2.3.

$find :: (a \Rightarrow r) \rightarrow (Tree\ a \Rightarrow r, Tree\ a) \rightarrow Maybe\ a \rightarrow (Maybe\ (a \Rightarrow r), Maybe\ a)$
 $find\ neg\ (y, Leaf)\ pos = (Just\ neg, pos)$
 $find\ neg\ (y, Node\ l\ a\ r)\ pos$
 $\quad | p\ a = find\ neg\ (Node_1\ y\ a\ r, l)\ (Just\ a)$
 $\quad | otherwise = find\ (Node_2\ l\ y\ r)\ (Node_3\ l\ a\ y, r)\ pos$

Figure 5 shows a run of the algorithm aligning it to exponential search on arrays.

The running time of *exponential-search* is $\Theta(\log i)$, where *i* is the position of the tipping point. This compares favourably to binary search, which is logarithmic in the size of the entire sequence. However, there is also a downside: due to the two-step process, exponential search is slower by a factor of two when the tipping point is closer to the rear.

4.4 Longest Uptrend – une dernière fois

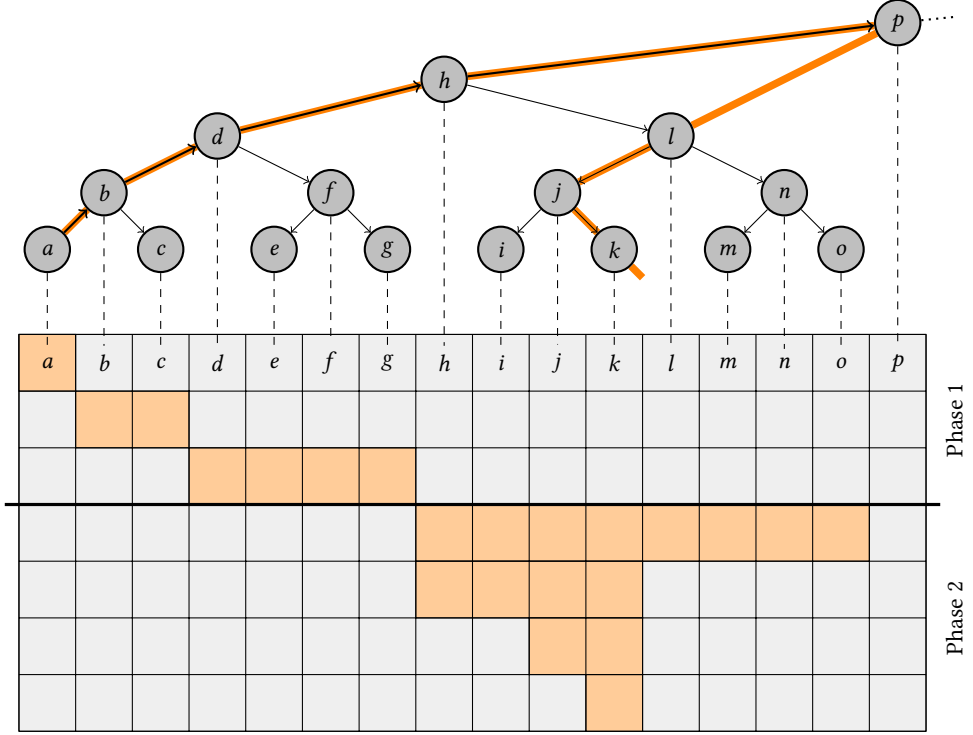
The step function replaces binary search by exponential search.

$insert :: (Ord\ a) \Rightarrow a \rightarrow Spine\ [a] \rightarrow Spine\ [a]$
 $insert\ a\ s = case\ exponential-search\ (a <) s\ of$
 $\quad (Nothing, Just\ v) \rightarrow cons\ (a : v)\ s$
 $\quad (Just\ f, Just\ v) \rightarrow apply\ f\ (a : v)$

As before, it distinguishes two cases: either a new uptrend is added at the front, or an existing uptrend is updated. Thanks to the improved interface, the code is now short and sweet.

Turning to the analysis of the running time, there are two worst-case scenarios: (1) either the to-be-inserted element is *consed* to the front, when the spine is a sequence of *Twos*; (2) or the element is added close to the rear. In both cases, the running time of *insert* is $\Theta(\log m)$, where *m* is

To illustrate the correspondence between finger trees and exponential search, consider the sequence $[a, b, \dots, p, \dots]$ together with the predicate $(l \leq)$. In this case, (k, l) is the unique tipping point. The diagram below shows an initial segment of the finger tree for $[a, b, \dots, p, \dots]$ and the corresponding array.



The path taken by *exponential-search* is highlighted in orange. In the first phase, the search explores the spine up to the element p . In the second phase, a binary search is conducted on the tree between h and p . The matrix below illustrates exponential search on the corresponding array. Each row represents a step of the algorithm. Again, there are two phases: First, a search range is determined by repeatedly doubling the candidate interval, then a binary search is performed by repeatedly halving the range. (Just in case you wonder, the second phase is *not* a standard (three-way) binary search, as the search always ends at a leaf rather than at a node containing an element.)

Fig. 5. Correspondence of finger trees and exponential search.

the length of a longest uptrend. So the final implementation of *best-longest-uptrend* is asymptotically optimal, as well.

In addition, the algorithm shows a nice *adaptive* behaviour: it finishes in linear time if the sequence is sorted or nearly sorted, for example, for inputs $[1, 2, \dots, n]$ and $[2, 1, 4, 3, 6, 5, \dots, n, n-1]$. In these cases, exponential search shows its strength — recall that it runs in $\Theta(\log i)$, where i is the position of the tipping point. This also speeds up the invocation of *apply*, as the zipping function simply walks back along the search path. Moreover, *cons* runs in amortized constant time.

It is also worth noting that the algorithm outperforms the imperative implementation for many inputs. For example, the imperative solution requires $\Theta(n \cdot \log n)$ time for the second example above, as a binary search is performed at every second step. Its logic is, in some sense, too simplistic:

it only works efficiently when new uptrends are added frequently, that is, when the **then** branch is entered frequently, see Figure 1.

Enough asymptotics, let the numbers speak for themselves: Appendix A shows the results of some micro-benchmarks. Overall, the benchmarks support the results of the asymptotic analysis. Quite pleasingly, the purely functional solutions are on a par with the imperative implementation; in some cases they even outperform the latter thanks to the improved adaptive behaviour. Hurrah!

5 Conclusion

All in all, this was an enjoyable exercise in algorithm, data structure, and interface design. Our first attempts at purely functional solutions to the “longest uptrend” problem focused on implementing *searchable*, *indexable* sequence types, supporting the following interface.

$$\begin{aligned} \text{Sequ} &:: \text{Type} \rightarrow \text{Type} \\ \text{nil} &:: \text{Sequ } a \\ \text{cons} &:: a \rightarrow \text{Sequ } a \rightarrow \text{Sequ } a \\ (!) &:: \text{Sequ } a \rightarrow \text{Int} \rightarrow a \\ \text{update} &:: \text{Sequ } a \rightarrow \text{Int} \rightarrow a \rightarrow \text{Sequ } a \\ \text{search} &:: (a \rightarrow \text{Bool}) \rightarrow \text{Sequ } a \rightarrow \text{Int} \end{aligned}$$

We tried out several data structures, but the efforts did not pay off. Simply replacing ephemeral arrays with these persistent sequence types worsened the running time by an order of magnitude.

Abstracting slightly from the details of our application (renaming *Spine* to *Sequ* and $a \Rightarrow \text{Spine } a$ to $\text{Pos } a$), our finger trees can be used to implement a radically different interface.

$$\begin{aligned} \text{Sequ} &:: \text{Type} \rightarrow \text{Type} \\ \text{nil} &:: \text{Sequ } a \\ \text{cons} &:: a \rightarrow \text{Sequ } a \rightarrow \text{Sequ } a \\ \text{search} &:: (a \rightarrow \text{Bool}) \rightarrow \text{Sequ } a \rightarrow (\text{Maybe } (\text{Pos } a, a), \text{Maybe } (\text{Pos } a, a)) \\ \text{Pos} &:: \text{Type} \rightarrow \text{Type} \\ \text{apply} &:: \text{Pos } a \rightarrow a \rightarrow \text{Sequ } a \end{aligned}$$

The revised interface is much more restrictive, but at the same time also much safer. The indices, the numerical positions, have disappeared from the scene — no “index out of range” errors anymore. Indices are replaced by paths. The function *search* returns two positions; each of these positions can be “applied” to a value to update the original sequence, replacing the general update mechanism based on numerical positions. This interface works beautifully for the “longest uptrend” problem. Whether it has other uses remains to be seen.

We have also gained some algorithmic insights. An important part of the lazy functional DNA is the idea of turning control structures into data structures: binary trees are the manifestation of binary search. Maybe this is old news to you, but it was new to us that finger trees are the incarnation of exponential search.

In conclusion, it is pleasing to see that we can keep up with or even surpass the imperative program. But, what was the crucial step in achieving this goal? We say the introduction of Huet’s zipper! This enabled us to organize the interplay of search, look-up, and update much more neatly. So, three cheers to Gérard!

Acknowledgments

We thank the anonymous reviewers for their constructive feedback, which has greatly enhanced the clarity and accessibility of the paper, and Jack Liell-Cock for carefully proof-reading an earlier draft of this manuscript.

A Appendix: Benchmarks

To evaluate the performance of our zipper-based implementations, we benchmark them against a few alternatives. In total, we compare six different implementations.

- **ST-ARRAYS:** The imperative solution, see Figure 1, transliterated into Haskell's imperative sublanguage, using the *ST*-monad and *ST*-arrays. This implementation serves as the baseline for comparison.
- **ST-ARRAYS-EXPONENTIAL-SEARCH:** An adaptation of the imperative solution, that uses exponential search instead of binary search to determine the tipping point.
- **RANDOM-ACCESS-LISTS:** A functional variant of the imperative solution, using persistent sequences based on node-oriented random-access lists. Look-up, update, and search are supported in logarithmic time, so the implementation is asymptotically optimal.
- **ADwH:** An implementation taken from the textbook *Algorithm Design with Haskell* [1], which is developed in a series of exercises (11.6–11.8). The solution is based on AVL-trees. Interestingly, it also avoids indices: In the update step, the AVL-tree is split at the tipping point and then merged with the new element replacing the old.
- **ZIPPER-RED-BLACK-TREES:** See Section 3.
- **ZIPPER-FINGER-TREES:** See Section 4.

We test the implementations on five different types of inputs to highlight the strengths and weaknesses of the approaches:

- $[1..n]$: The adaptive implementations achieve linear running time for sorted inputs.
- *interleave* $[2..n+1] [1..n]$: The methods that use exponential search finish in linear time, whereas the imperative solution requires linear-logarithmic time.
- *replicate* $n \text{ } n \text{ } ++ [1..n]$: As exponential search takes twice as long to locate elements at the rear of the candidate sequence, this input represents a worst-case scenario for ST-ARRAYS-EXPONENTIAL-SEARCH, RANDOM-ACCESS-LISTS, and ZIPPER-FINGER-TREES.
- *replicate* $n \text{ } 1 \text{ } ++ [1..n]$: This is a best-case scenario for exponential search as the tipping point is always on the far left.
- **Random list:** To estimate the average performance, we test on a random list of length n with elements in the range $(1, n)$.

The results are presented in Figure 6, where running times (in seconds) are plotted against the lengths of the input lists (scaled by 5,000,000). The code, which is publicly available [4], was compiled using GHC 9.10.1 with the `-O2` optimization flag turned on. The benchmarks were executed on an AMD Ryzen 9 9950X CPU.

The data shows that the zipper-based implementations perform on par with the imperative programs. When adaptivity is advantageous, finger trees outperform the imperative approach. In the case of random inputs, finger trees perform worse than red-black trees due to the additional overhead of the exponential search. Note that node-oriented random-access lists are structurally similar to the finger trees presented in this paper, which is why RANDOM-ACCESS-LISTS, ZIPPER-FINGER-TREES, and ST-ARRAYS-EXPONENTIAL-SEARCH exhibit the same performance characteristics.

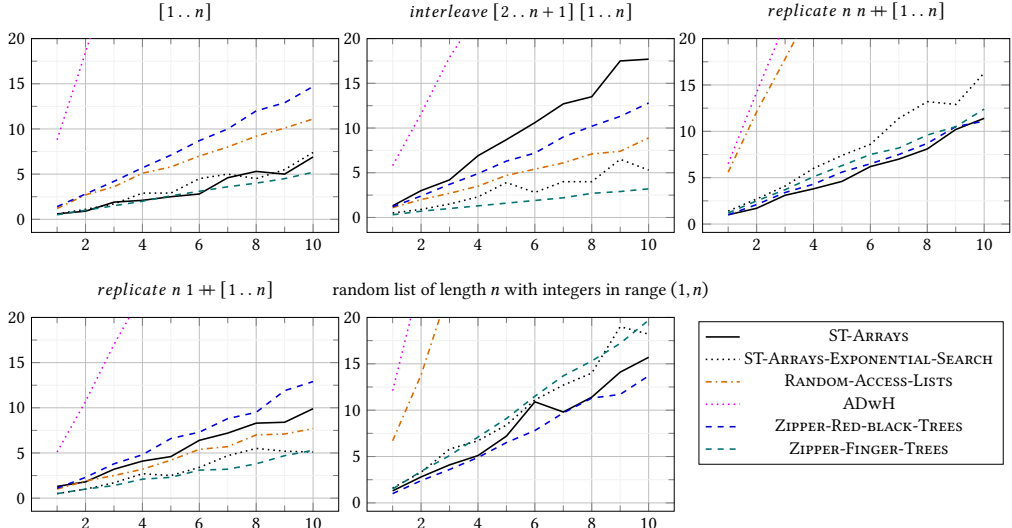


Fig. 6. Performance comparison of different solutions to the longest uptrend problem

References

- [1] Richard Bird and Jeremy Gibbons. 2020. *Algorithm Design with Haskell*. Cambridge University Press.
- [2] Edsger Wybe Dijkstra. 1988. *A method of programming*. Addison-Wesley.
- [3] Alexander Dinges and Ralf Hinze. 2025. Binary search—think positive. *Journal of Functional Programming* 35 (2025). doi:10.1017/S0956796825000061
- [4] Alexander Dinges and Ralf Hinze. 2025. *Truly Functional Solutions to the Longest Uptrend Problem (Functional Pearl) Artifact*. doi:10.5281/zenodo.15721901
- [5] David Gries. 1981. *The Science of Programming*. Springer-Verlag New York. doi:10.1007/978-1-4612-5983-1
- [6] Ralf Hinze. 1999. Constructing Red-Black Trees. In *Proceedings of the Workshop on Algorithmic Aspects of Advanced Programming Languages (WAAAPL'99)* (Paris, France, 30 September 1999), Chris Okasaki (Ed.), 89–99. The proceedings appeared as a technical report of Columbia University, CUCS-023-99.
- [7] Gérard Huet. 1997. Functional Pearl: The Zipper. *Journal of Functional Programming* 7, 5 (September 1997), 549–554.
- [8] Conor McBride. 2001. The Derivative of a Regular Type is its Type of One-Hole Contexts. <https://api.semanticscholar.org/CorpusID:15235160>
- [9] Chris Okasaki. 1999. Functional Pearl: Red-black trees in a functional setting. *Journal of Functional Programming* 9, 4 (July 1999), 471–477.

Received 2025-02-27; accepted 2025-06-27